

Day 6: Docker Basics, Containerised LLM Deployment & Model Versioning

Week 1 — Local Inference to Containerised Serving

Goal: Ship your LLM server as a reproducible, versioned Docker image. Write a multi-stage Dockerfile for the FastAPI inference server, mount model weights as a volume (never bake them into the image), run with --gpus all, tag images by model name and quantisation level, and implement /health and /version endpoints for production-grade rollback capability.

PART A — Docker Basics for ML Engineers

Why Containers for LLM Serving?

A Python virtual environment is reproducible on one machine. A Docker container is reproducible on any machine that runs Docker. For LLM serving this matters because CUDA drivers, Python versions, torch builds, and flash-attn wheel compatibility are notoriously fragile. A container freezes the entire software stack -- OS libraries, CUDA toolkit, Python packages, and your application code -- into a single artefact that runs identically in development, staging, and production.

Core Docker Concepts

Concept	Definition	LLM Example
Image	Read-only snapshot of filesystem + metadata	pytorch/pytorch:2.1-cuda12.1 + your app code
Container	Running instance of an image	A live vLLM or FastAPI process
Layer	Incremental filesystem change stored in the image	pip install transformers adds one layer
Volume	Directory mounted from host into container at runtime	models mounted to /root/.cache/huggingface
Registry	Remote store for images (push/pull)	Docker Hub, AWS ECR, GCR
Dockerfile	Recipe for building an image, step by step	FROM, RUN, COPY, CMD instructions
Multi-stage	Multiple FROM blocks -- only final stage shipped	Builder stage installs deps, runtime is slim

Critical Rule: Never Bake Model Weights Into the Image

LLM weights are 2-150+ GB. Baking them into a Docker image means: the image is enormous and slow to push/pull, every model update requires rebuilding and pushing the entire image, and you cannot share the same image across model versions. The correct pattern is: image contains only code and dependencies, weights live on a volume (host filesystem, NFS, or object store) and are mounted at container start.

A.1 Single-Stage Dockerfile (Simple, Development)

A.2 Multi-Stage Dockerfile (Production -- Lean Image)

```
# Dockerfile -- multi-stage build for production LLM serving
# Stage 1 (builder): install all build tools and compile dependencies
# Stage 2 (runtime): copy only what's needed to run -- no compilers

# ■■■ Stage 1: Builder ████████████████████████████████████████████████████████████████████████████
FROM pytorch/pytorch:2.1.0-cuda12.1-cudnn8-devel AS builder

WORKDIR /build

# Install build tools needed for flash-attn wheel compilation
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gcc g++ ninja-build && \
    rm -rf /var/lib/apt/lists/*

# Copy and install Python dependencies into a prefix directory
```


PART B — Containerised LLM Deployment with Docker Compose

B.1 Docker Compose for Local Multi-Service Deployment

```
# docker-compose.yml -- complete local LLM serving stack
version: "3.8"

services:

  llm-server:
    image: myregistry.io/llm/llama3-8b:fp16-v1.0.2
    build:
      context: .
      dockerfile: Dockerfile
      target: runtime
    ports:
      - "8080:8080"
    environment:
      # Model config -- override to switch models without rebuilding
      MODEL_ID: "meta-llama/Meta-Llama-3-8B-Instruct"
      PORT: "8080"
      LOG_LEVEL: "info"
      HF_HUB_OFFLINE: "1"          # use only cached weights
      # Torch settings
      CUDA_VISIBLE_DEVICES: "0"
      PYTORCH_CUDA_ALLOC_CONF: "max_split_size_mb:512"
    volumes:
      # Mount pre-downloaded model weights -- NEVER bake into image
      - model_cache:/root/.cache/huggingface
    deploy:
      resources:
        reservations:
          devices:
            - capabilities: [gpu]
              device_ids: ["0"]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 120s    # allow time for model load
    restart: unless-stopped
    logging:
      driver: "json-file"
      options:
        max-size: "100m"
        max-file: "5"

# Optional: Nginx reverse proxy (useful for SSL, rate limiting)
nginx:
  image: nginx:alpine
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
```


PART C — Model Versioning, /health, /version & Rollback

Tagging Strategy for LLM Images

Unlike application code where SemVer (1.2.3) is standard, LLM image tags must encode three orthogonal dimensions: the base model identity, the quantisation/precision level, and the application code version. A well-designed tag makes the contents of the image self-evident and enables targeted rollbacks without confusion.

Tag Component	Example Values	Purpose
Base model name	llama3-8b, phi3-mini, mistral-7b	Which weights the image serves
Precision/quant	fp16, awq-int4, gguf-q4km	VRAM requirement, quality level
App version	v1.0.2 (SemVer of your server code)	Which FastAPI/server code version
Full tag	llama3-8b:awq-int4-v1.0.2	Unambiguous, rollback-safe reference
Latest alias	llama3-8b:awq-int4-latest	Convenience -- never use in production

C.1 /health and /version Endpoints

```
# server.py -- production health and version endpoints
from fastapi import FastAPI
from pydantic import BaseModel
import torch, time, os, platform

app = FastAPI()

# Build metadata injected at image build time via --build-arg
APP_VERSION   = os.getenv("APP_VERSION",   "unknown")
MODEL_ID      = os.getenv("MODEL_ID",      "unknown")
QUANT_LEVEL   = os.getenv("QUANT_LEVEL",   "fp16")
BUILD_COMMIT  = os.getenv("BUILD_COMMIT",  "unknown") # git SHA
BUILD_TIME    = os.getenv("BUILD_TIME",    "unknown")

# Track server start time for uptime calculation
SERVER_START  = time.time()
model_loaded  = False # set to True after model loads in startup event

class HealthResponse(BaseModel):
    status: str # "ok" | "degraded" | "starting"
    model_loaded: bool
    uptime_s: float
    gpu_available: bool
    gpu_name: str
    vram_used_gb: float
    vram_total_gb: float

class VersionResponse(BaseModel):
    app_version: str
    model_id: str
    quant_level: str
```



```

    build_commit: str
    build_time: str
    python_version: str
    torch_version: str
    cuda_version: str

@app.get("/health", response_model=HealthResponse)
async def health():
    gpu_avail = torch.cuda.is_available()
    vram_used = torch.cuda.memory_allocated() / 1e9 if gpu_avail else 0
    vram_total = torch.cuda.get_device_properties(0).total_memory / 1e9 if gpu_avail else 0
    gpu_name = torch.cuda.get_device_name(0) if gpu_avail else "none"

    return HealthResponse(
        status = "ok" if model_loaded else "starting",
        model_loaded = model_loaded,
        uptime_s = round(time.time() - SERVER_START, 1),
        gpu_available= gpu_avail,
        gpu_name = gpu_name,
        vram_used_gb = round(vram_used, 2),
        vram_total_gb= round(vram_total, 2),
    )

@app.get("/version", response_model=VersionResponse)
async def version():
    return VersionResponse(
        app_version = APP_VERSION,
        model_id = MODEL_ID,
        quant_level = QUANT_LEVEL,
        build_commit = BUILD_COMMIT,
        build_time = BUILD_TIME,
        python_version = platform.python_version(),
        torch_version = torch.__version__,
        cuda_version = torch.version.cuda or "N/A",
    )

# Readiness probe: stricter than liveness -- fails until model is loaded
@app.get("/ready")
async def ready():
    if not model_loaded:
        from fastapi import HTTPException
        raise HTTPException(503, "Model not yet loaded")
    return {"ready": True}

```

C.2 Injecting Build Metadata into the Image

```

# Updated Dockerfile -- inject build metadata as image labels and env vars
FROM pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime AS runtime

# Build arguments passed in at build time
ARG APP_VERSION=dev
ARG BUILD_COMMIT=unknown
ARG BUILD_TIME=unknown
ARG MODEL_ID=gpt2
ARG QUANT_LEVEL=fp16

# Bake into image as labels (visible in docker inspect)

```

```

LABEL org.opencontainers.image.version=$APP_VERSION
LABEL org.opencontainers.image.revision=$BUILD_COMMIT
LABEL org.opencontainers.image.created=$BUILD_TIME
LABEL llm.model.id=$MODEL_ID
LABEL llm.model.quant=$QUANT_LEVEL

# Also expose as env vars for runtime access by the application
ENV APP_VERSION=$APP_VERSION
ENV BUILD_COMMIT=$BUILD_COMMIT
ENV BUILD_TIME=$BUILD_TIME
ENV MODEL_ID=$MODEL_ID
ENV QUANT_LEVEL=$QUANT_LEVEL

WORKDIR /app
COPY --from=builder /install /usr/local
COPY server.py utils.py ./

EXPOSE 8080
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8080"]

# Build with metadata
docker build \
  --build-arg APP_VERSION=1.0.2 \
  --build-arg BUILD_COMMIT=$(git rev-parse --short HEAD) \
  --build-arg BUILD_TIME=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --build-arg MODEL_ID=meta-llama/Meta-Llama-3-8B-Instruct \
  --build-arg QUANT_LEVEL=fp16 \
  -t myregistry.io/llm/llama3-8b:fp16-v1.0.2 .

# Verify labels were baked in
docker inspect myregistry.io/llm/llama3-8b:fp16-v1.0.2 | \
  python -m json.tool | grep -A 20 '"Labels"'

```

[illegible]

```
# ■■ Step 5: Verify health after rollback
sleep 120 # wait for model load
curl http://production-server/health | python -m json.tool
curl http://production-server/version | python -m json.tool
# Confirm app_version == "1.0.2"

# ■■ Step 6: Document the rollback in your incident log
echo "$(date -u) ROLLBACK: llama3-8b fp16-v1.0.3 -> fp16-v1.0.2 (reason: ...)\" \
    >> /var/log/llm-deploys.log
```

Testing & Metrics

Container Build and Run Verification

```
# 1. Build and verify image size
docker build -t llm-server:test .
docker images llm-server:test
# Runtime stage should be ~8-12 GB (PyTorch base + packages); weights NOT included

# 2. Check image layer sizes (find bloated layers)
docker history llm-server:test --no-trunc | head -20

# 3. Run container and verify GPU access
docker run --gpus all --rm llm-server:test python -c "
import torch
print('CUDA:', torch.cuda.is_available())
print('GPU:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'N/A')
print('VRAM:', torch.cuda.get_device_properties(0).total_memory // 1e9, 'GB')
"

# 4. Start server with volume-mounted weights and verify endpoints
docker run --gpus all -d \
    -p 8080:8080 \
    -v /data/models:/root/.cache/huggingface \
    -e MODEL_ID=gpt2 \
    --name llm-test \
    llm-server:test

# Wait for model load then test
sleep 30
curl http://localhost:8080/health      | python -m json.tool
curl http://localhost:8080/version    | python -m json.tool
curl http://localhost:8080/ready      | python -m json.tool

# 5. Test docker-compose healthcheck
docker compose up -d
docker compose ps      # should show "healthy" after start_period
docker compose logs llm-server | tail -20

# 6. Verify HF_HUB_OFFLINE prevents downloads
docker run --rm -e HF_HUB_OFFLINE=1 \
    -v /data/models:/root/.cache/huggingface \
    llm-server:test python -c "
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained('gpt2')
"
```

```
print('Loaded from cache successfully')
"
```

```
# Cleanup
docker rm -f llm-test
```

Key Metrics for Container Health

Metric	Command	Target
Image size	docker images llm-server:tag	< 15 GB (code only, no weights)
Container startup	time docker run ... --rm llm-server:tag python -c 'import torch'	< 30s
Model load time	/health uptime_s when status changes to 'ok'	< 120s for 8B FP16
VRAM after load	/health vram_used_gb	Matches expected from formula
Health check pass	docker compose ps -- column Status	healthy (not starting/unhealthy)
Rollback time	docker compose up -d + /health ok	< 3 min for same image tag

Key Takeaways

- **Multi-stage builds** keep production images lean: the builder stage installs compilers and build tools; the runtime stage copies only the compiled packages. Saves 2-5 GB on a typical ML image.
- **Never bake model weights into the image.** Always mount as a volume. A 16 GB image with weights embedded means every code change requires pushing 16 GB to the registry.
- **Tag images with model + quant + version:** llama3-8b:awq-int4-v1.0.2 is unambiguous. Never use :latest in production -- you lose rollback safety.
- **Inject build metadata as ARGs/labels:** app version, git commit, build timestamp, model ID, and quant level should all be readable from docker inspect and from the /version endpoint at runtime.
- **/health, /ready, and /version are required endpoints:** /health for liveness (is the process alive?), /ready for readiness (is the model loaded?), /version for audit (what is actually running?).
- **Document rollback steps before you need them.** The rollback procedure is: identify last good tag, pull it, update compose file, restart service, verify /version confirms the rollback.

Next topic: **Day 7 -- Basic Observability + Week 1 Review:** instrument FastAPI with Prometheus, build a Grafana latency dashboard, define TTFT/TBT/E2E latency, and baseline your stack's throughput.